

Estrutura de Software — BMS

Battery Management System com TLE9012DQU + TLE9015DQU + STM32G491RET6

Documento Técnico

Rev. 1.5

2026-08-25

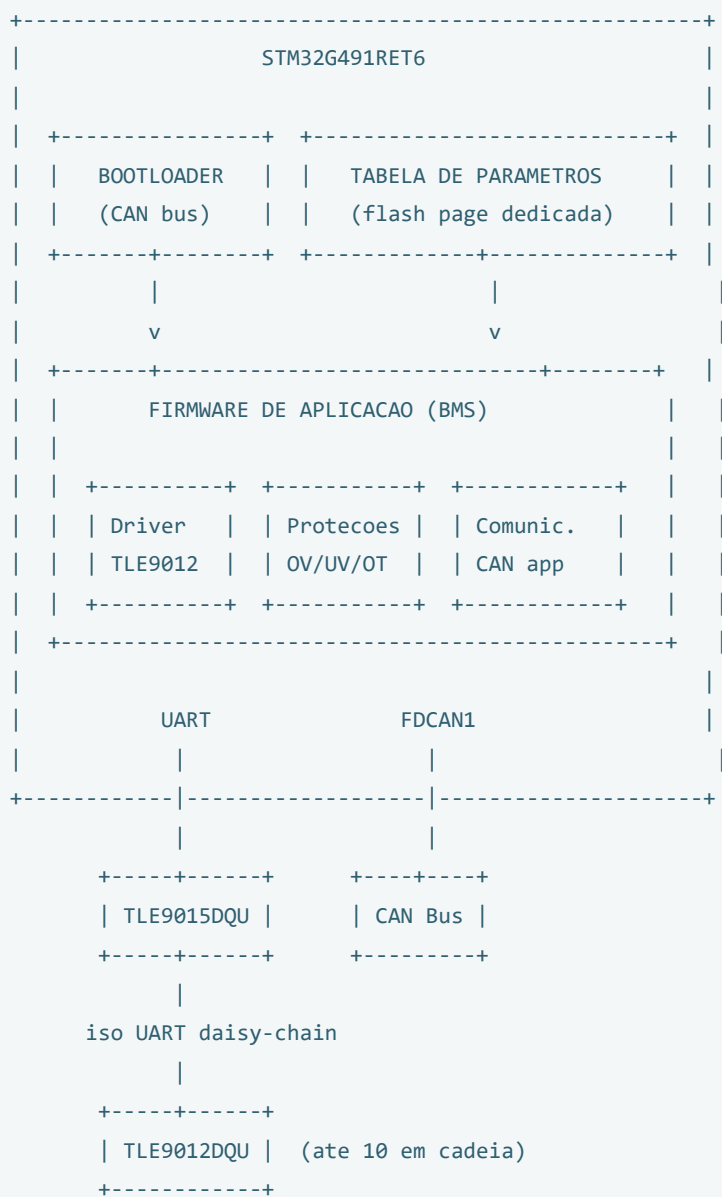
Equipe: AMPERA • **Autor:** Guilherme Lettmann

SUMÁRIO

1. Visão Geral da Arquitetura
 1. Hardware de Desenvolvimento
 2. Fases de Implementação
2. Mapa de Memória Flash
3. Bootloader CAN
4. Tabela de Parâmetros
5. Firmware de Aplicação
 1. Módulos
 2. Máquina de Estados
6. Driver TLE9012 / TLE9015
 1. Protocolo iso UART
 2. Estrutura do Frame
 3. CRC SAE-J1850
 4. Sequência de Inicialização
 5. Leitura de Tensões
 6. Leitura de Temperatura
 7. Portabilidade — Isolamento do Transporte
 8. Configuração do Periférico no STM32CubeMX
7. Estrutura de Diretórios do Projeto
8. Fluxo de Atualização em Campo
9. Validação em Bancada
10. Diagnóstico e Proteções
 1. Código de falha
 2. Bits do GEN_DIAG
 3. Máscaras por célula
 4. Histerese
 5. Injeção de falha
 6. Sinalização no LD2
11. Referências
 1. Uso da Biblioteca de Referência

1. VISÃO GERAL DA ARQUITETURA

O software do BMS é dividido em três camadas independentes, armazenadas em regiões distintas da flash do STM32G491RET6:



Objetivo principal: permitir que mudanças de configuração (número de células, thresholds, IDs CAN, etc.) sejam feitas **apenas alterando parâmetros**, sem recompilação. Atualizações de firmware são feitas via **bootloader CAN**, eliminando a necessidade de programador físico.

1.1 Hardware de Desenvolvimento

ITEM	COMPONENTE	OBSERVAÇÃO
MCU	NUCLEO-G491RE	Cortex-M4F @170 MHz, 512 KB flash, 128 KB RAM, 3× FDCAN, ST-LINK integrado
Transceiver	AURIX TLE9015 Adapter Board V2.0	Sinais acessíveis por test points / header COM6 — não exige AURIX

ITEM	COMPONENTE	OBSERVAÇÃO
Sensing	BMS Balancing and Sensing Board V6	TLE9012DQU + 12 canais + ladder resistivo on-board
Debug	ST-LINK (on-board)	SWD + VCP para <code>printf</code> pelo mesmo cabo USB

NOTA — ESCOLHA DO MCU

O STM32G491RET6 é equivalente ao STM32G474RET6 para esta aplicação: mesma família STM32G4, mesmo core, mesma flash e mesmo número de FDCAN. A diferença relevante é o HRTIM, presente apenas no G474 e sem uso neste projeto. Um AURIX TC375 foi considerado (ASIL-D, lockstep, tri-core), mas descartado para a fase de bring-up: a gravação de flash no AURIX é substancialmente mais complexa (bursts de página, rotinas executadas de PSPR, UCBs), o que encarece o bootloader CAN sem benefício funcional para 12 células. A opção permanece aberta — ver seção 6.7.

1.2 Fases de Implementação

O bootloader é a última etapa, não a primeira. Até lá, gravação e depuração são feitas normalmente pelo ST-LINK.

FASE	ESCOPO	ENDEREÇO DO APP	GRAVAÇÃO
1	Driver TLE9012 — ler as 12 tensões via UART	<code>0x0800 0000</code>	ST-LINK (SWD)
2	Proteções, temperatura, balanceamento	<code>0x0800 0000</code>	ST-LINK (SWD)
3	Tabela de parâmetros em flash + CAN da aplicação	<code>0x0800 0000</code>	ST-LINK (SWD)
4	Bootloader CAN	<code>0x0800 4000</code>	ST-LINK <i>ou</i> CAN

NOTA

A migração da fase 3 para a 4 exige apenas duas mudanças no app: `ORIGIN` do linker script para `0x08004000` e reconfiguração do `VTOR`. O ST-LINK continua gravando e depurando o app normalmente mesmo com o bootloader presente na flash — as duas vias de gravação coexistem.

2. MAPA DE MEMÓRIA FLASH

O STM32G491RET6 possui 512 KB de flash. O tamanho de página depende do bit de opção `DBANK` (configuração single-bank ou dual-bank) — consultar RM0440 para o valor efetivo antes de implementar o driver de flash. O layout abaixo usa regiões múltiplas de 4 KB, portanto permanece válido em qualquer uma das duas configurações.

REGIÃO	ENDEREÇO INICIAL	TAMANHO	DESCRIÇÃO
BOOTLOADER	<code>0x0800 0000</code>	16 KB	Bootloader CAN — protegido contra escrita (WRP)
APP	<code>0x0800 4000</code>	480 KB	Firmware de aplicação BMS
PARAMS	<code>0x0807 C000</code>	8 KB	Tabela de parâmetros (página ativa)
PARAMS_BKP	<code>0x0807 E000</code>	8 KB	Tabela de parâmetros (backup)

NOTA

O bootloader deve ser protegido com Write Protection (WRP) via option bytes antes de ir a campo, para evitar sobrescrita acidental. Durante o desenvolvimento (fases 1–3) a WRP fica desabilitada.

ATENÇÃO

Nas fases 1 a 3 o app é compilado no endereço default `0x0800 0000` e ocupa a flash inteira — o mapa acima só passa a valer na fase 4, quando o bootloader é introduzido.

3. BOOTLOADER CAN

O bootloader reside permanentemente na flash e é executado no reset. Ele decide se entra em modo de atualização ou pula para a aplicação.

3.1 Condições de Entrada no Bootloader

1. **Flag em RAM persistente** — a aplicação seta um valor mágico (ex: `0xDEADBEEF`) num endereço reservado de RAM e executa reset por software. O bootloader lê esse endereço; se encontrar o valor mágico, entra em modo de atualização.
2. **Aplicação inválida** — o bootloader verifica o CRC32 da região APP armazenado no último word da imagem. Se inválido, permanece no bootloader.
3. **Comando CAN** — durante uma janela de 200 ms após o reset, o bootloader escuta uma mensagem CAN de solicitação de atualização.

3.2 Protocolo de Atualização

PASSO	DIREÇÃO	CAN ID	CONTEÚDO
1	PC → MCU	<code>0x600</code>	Comando ENTER_BOOTLOADER
2	MCU → PC	<code>0x601</code>	ACK + versão do bootloader
3	PC → MCU	<code>0x600</code>	ERASE_APP (apaga região APP)
4	MCU → PC	<code>0x601</code>	ACK após erase completo
5	PC → MCU	<code>0x602</code>	DATA — pacotes de 8 bytes sequenciais
6	MCU → PC	<code>0x601</code>	ACK a cada página (4 KB) gravada
7	PC → MCU	<code>0x600</code>	VERIFY — CRC32 da imagem completa
8	MCU → PC	<code>0x601</code>	ACK/NACK — resultado da verificação
9	PC → MCU	<code>0x600</code>	JUMP_APP — reset para a aplicação

ATENÇÃO

O FDCAN1 do STM32G491 é usado tanto pelo bootloader quanto pela aplicação. O bootloader configura o CAN com bitrate fixo de 500 kbit/s. A aplicação pode reconfigurar conforme a tabela de parâmetros.

3.3 Salto para a Aplicação

Após validar a imagem, o bootloader:

- 1. Desabilita todas as interrupções
- 2. Desinicializa periféricos usados (FDCAN, SysTick)
- 3. Configura o MSP (Main Stack Pointer) com o valor do primeiro word em 0x0800 4000
- 4. Salta para o endereço do Reset_Handler em 0x0800 4004

REQUISITO

O linker script da aplicação deve ter FLASH (rx) : ORIGIN = 0x08004000 e o VTOR deve ser reconfigurado no SystemInit para 0x08004000.

4. TABELA DE PARÂMETROS

Todos os valores configuráveis do BMS ficam numa struct armazenada nas últimas páginas da flash. A aplicação lê essa struct no boot e nunca usa constantes hard-coded.

PARÂMETRO	TIPO	DEFAULT	UNIDADE	DESCRIÇÃO
num_cells	uint8	12	—	Número de células por TLE9012
num_slaves	uint8	1	—	Número de TLE9012 na cadeia
ov_threshold_mV	uint16	4200	mV	Threshold de sobretensão
uv_threshold_mV	uint16	2800	mV	Threshold de subtensão
ot_threshold_C	int16	60	°C	Threshold de sobretemperatura
ut_threshold_C	int16	-20	°C	Threshold de subtemperatura
bal_enable	uint8	1	bool	Habilita balanceamento passivo
bal_delta_mV	uint16	10	mV	Delta para iniciar balanceamento
can_bitrate	uint32	500000	bit/s	Bitrate do CAN da aplicação
can_base_id	uint16	0x100	—	ID base das mensagens CAN TX
meas_period_ms	uint16	100	ms	Período de medição de tensão
temp_period_ms	uint16	1000	ms	Período de medição de temperatura
num_ntc	uint8	5	—	Número de sensores NTC por TLE9012
uart_baudrate	uint32	2000000	baud	Baudrate da iso UART — mínimo 1000000
crc32	uint32	—	—	CRC32 da struct (validação)

NOTA

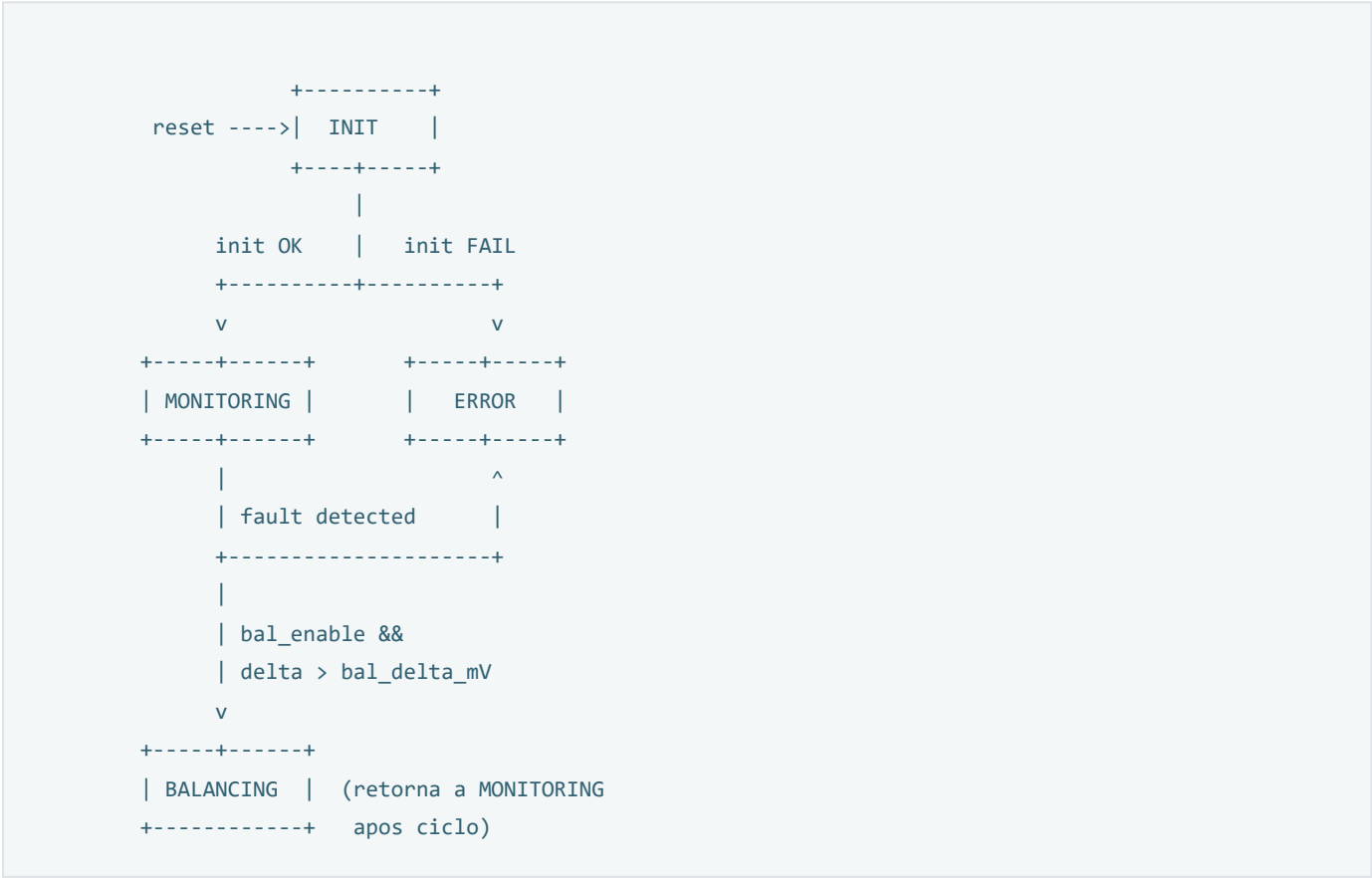
Os parâmetros podem ser atualizados via mensagem CAN sem necessidade de bootloader — a aplicação recebe o comando, apaga a página de parâmetros, grava a nova struct e reinicia. O campo crc32 protege contra corrupção; se inválido, a aplicação carrega os valores default.

5. FIRMWARE DE APLICAÇÃO

5.1 Módulos

MÓDULO	ARQUIVO	RESPONSABILIDADE
Main	<code>main.c</code>	Init de periféricos, loop principal, scheduler
TLE9012 Driver	<code>tle9012.c/h</code>	Comunicação iso UART, leitura de células e temperaturas, balanceamento
TLE9015 Driver	<code>tle9015.c/h</code>	Init do transceiver, wake-up, sleep
CRC	<code>crc_j1850.c/h</code>	CRC8 SAE-J1850 e CRC3 do reply frame, bit a bit
Parâmetros	<code>params.c/h</code>	Leitura/escrita da tabela de parâmetros na flash
Proteções	<code>protection.c/h</code>	Lógica OV, UV, OT, UT, open-wire — flags e ações
Balanceamento	<code>balancing.c/h</code>	Algoritmo de balanceamento passivo baseado em delta de tensão
CAN App	<code>can_app.c/h</code>	Transmissão de dados, recepção de comandos, atualização de parâmetros
BSP	<code>bsp.c/h</code>	Abstração de GPIOs, LEDs, ERR pin

5.2 Máquina de Estados



INIT: configura periféricos, carrega parâmetros, inicializa cadeia TLE9012 (atribui `NODE_IDs`), configura registradores (`PART_CONFIG`, `OV/UV thresholds`, `TEMP_CONF`).

MONITORING: loop periódico — dispara medição (`PCVM_START`), lê 12 tensões e temperaturas, verifica proteções, transmite via CAN.

BALANCING: ativa balanceamento passivo nas células acima da tensão mínima + delta. Desativa durante medição (PBOFF).

ERROR: desativa balanceamento, sinaliza via CAN e ERR pin. Permite retry após comando CAN.

6. DRIVER TLE9012 / TLE9015

6.1 Protocolo iso UART

A comunicação entre o MCU e o TLE9015 é feita por UART padrão. O TLE9015 converte para iso UART (diferencial, half-duplex, 2 fios com acoplamento capacitivo ou por transformador) e encaminha para a cadeia de TLE9012.

PARÂMETRO	VALOR
Baudrate	2 Mbit/s (oficial) — mínimo 1 Mbit/s
Data bits	8
Paridade	Nenhuma
Stop bits	1
Direção	Single-wire half-duplex (TX e RX no mesmo pino)

REQUISITO CRÍTICO — BAUDRATE

A iso UART **não funciona abaixo de 1 Mbit/s**. O baudrate oficialmente suportado é 2 Mbit/s. Os 115200 baud que aparecem na Figura 14 do manual da Infineon referem-se ao **terminal de debug PC↔AURIX** (TeraTerm), *não* ao link iso UART — confundir os dois leva a um link que jamais estabelece comunicação. O próprio menu do firmware de demo confirma (`CS 1000000 → UART baudrate change to 1Mb`).

REQUISITO CRÍTICO — TIMING ENTRE BYTES

Os bytes de um frame devem ser transmitidos **sem atraso significativo entre si**, sob pena de disparar timeout no transceiver iso UART e corromper a mensagem. A 2 Mbit/s cada byte dura 5 µs (10 bits), o que inviabiliza envio byte-a-byte por polling. **Consequência de projeto: o TX deve usar DMA**, entregando os 6 bytes do frame como um bloco contíguo ao USART. O RX também deve usar DMA (circular) para não perder bytes.

NOTA — VIABILIDADE NO G491

A 170 MHz, 2 Mbit/s exige divisor `BRR = 85` no USART1 — valor inteiro exato, sem erro de baudrate. É justamente essa exigência de timing que inviabiliza Arduinos baseados em ATmega328 nesta aplicação, e que o Cortex-M4 com DMA atende sem dificuldade.

Ligação física MCU → TLE9015

Pelo esquemático do transceiver (Figura 24 do manual), `UART_LS` é um **pino único**: o host conecta TX e RX a ele através da rede `R_UART` (1,5 kΩ típico, $\tau \leq 50$ ns — Tabela 5). Trata-se portanto de **single-wire half-duplex**, e não de um par TX/RX independente.

A biblioteca de referência `TLE9012_Arduino_Lib` (ver seção 9) confirma independentemente a mesma topologia: “cada pino funciona como RX e TX”, sendo necessário “conectar os pinos rx e tx através de um resistor série”, com

valor típico de 1 kΩ a 1,5 kΩ — coerente com o `R_UART` da Tabela 5.

ATENÇÃO

Duas opções de implementação no STM32: (a) modo single-wire nativo do USART pelo bit `HDSEL` (`HAL_HalfDuplex_Init`), em que TX e RX compartilham o pino `TX` em open-drain; ou (b) manter TX e RX separados no MCU e uni-los externamente pelo resistor série, como fazem as bibliotecas Arduino. A opção (a) dispensa o resistor no lado do MCU, mas exige descartar o eco local de cada byte transmitido. Confirmar a topologia da placa antes de fiar.

SINAL (ADAPTER BOARD)	NUCLEO-G491RE	FUNÇÃO
UART_LS	PA9 – USART1_TX	iso UART low-side (single-wire, via R_UART)
nSLEEP	GPIO livre	Wake-up do transceiver
ERRQ	GPIO (EXTI)	Sinalização de erro do transceiver
3.3V / GND	3V3 / GND	Referência comum — lado de baixa tensão

NOTA

O USART2 (`PA2` / `PA3`) do NUCLEO-64 está ligado ao VCP do ST-LINK por solder bridges; reservá-lo para `printf` de debug permite acompanhar as tensões pelo mesmo cabo USB da gravação. Por isso o TLE9015 usa o USART1. A tabela acima é uma proposta de pinagem, não uma restrição do hardware.

REQUISITO

A NUCLEO-G491RE **não possui transceiver CAN on-board**. Para as fases 3 e 4 é necessário um transceiver externo (ex.: TJA1051, SN65HVD230) ligado aos pinos do FDCAN1, com terminação de 120 Ω.

6.2 Estrutura do Frame

O frame de **escrita** tem 6 bytes; o de **leitura** tem 4 (não possui campo DATA):

BYTE	CAMPO	WRITE	READ	DESCRIÇÃO
0	SYNC	✓	✓	Sempre <code>0x1E</code> — sincronização
1	ID	✓	✓	bit[7]: R/W (1=write, 0=read); bits[5:0]: NODE_ID (0x00=não endereçado, 0x3F=broadcast)
2	ADDR	✓	✓	Endereço do registrador alvo
3–4	DATA	✓	—	Dado, big-endian. Ausente no frame de leitura
5	CRC8	✓	✓	CRC SAE-J1850 sobre os bytes anteriores

CORREÇÃO EM RELAÇÃO ÀS REVISÕES 1.0–1.2

As revisões anteriores descreviam o frame de leitura como tendo 6 bytes, com o host enviando `0x0000` no campo DATA. Está **incorreto**: o frame de leitura tem 4 bytes. Confere com a Tabela 11 do manual (que lista apenas SYNC / ID / ADDR / CRC) e com a contagem de bytes da biblioteca de referência.

Os bytes trafegam com o **bit mais significativo primeiro**, ao contrário do UART convencional. A biblioteca Arduino contorna isso invertendo os bits por software (`msb_first_converter`) porque o core do Arduino só transmite LSB-first.

VANTAGEM DO STM32

CR2.MSBFIRST

O USART do STM32 implementa MSB-first em **hardware**, tanto no TX quanto no RX. No CubeMX: USART1 → Parameter Settings → Advanced Parameters → *MSB First* = *Enable*. Dispensa completamente a inversão de bits por software.

Eco e contagem de bytes

Como a linha é compartilhada, o host **recebe de volta tudo o que transmite**. O driver deve descartar o eco antes de interpretar a resposta:

OPERAÇÃO	HOST ENVIA	TOTAL RECEBIDO	COMPOSIÇÃO
Read	4 bytes	9 bytes	4 de eco + 5 de resposta
Write	6 bytes	7 bytes	6 de eco + 1 byte de reply frame

6.3 CRC — dois polinômios distintos

O protocolo usa **dois** CRCs diferentes, um para frames de dados e outro para o reply frame:

CRC	APLICAÇÃO	POLINÔMIO	INIT	XOR FINAL	VALIDAÇÃO
CRC8	Frames de dados (read e write)	0x1D	0xFF	0xFF	Comparar com o byte recebido
CRC3	Reply frame do write (1 byte)	0xB0	o próprio byte	—	Válido se o resultado for zero

O CRC8 é o SAE-J1850: $G(z) = z^8 + z^4 + z^3 + z^2 + 1$.

CORREÇÃO EM RELAÇÃO ÀS REVISÕES 1.0–1.3

As revisões anteriores recomendavam implementar o CRC por *lookup table* de 256 bytes, alegando aperto de tempo. **A justificativa estava errada.** Um frame tem no máximo 5 bytes, ou seja 40 iterações — cerca de 0,3 μs a 170 MHz, contra ~30 μs de duração do próprio frame no fio a 2 Mbit/s. A tabela custaria 256 bytes de flash para economizar 1% do tempo da transação. A implementação adotada é **bit a bit**, que além de suficiente é mais fácil de auditar.

O CRC3 opera sobre o único byte de reply, deslocando 5 vezes com realimentação condicional do polinômio `0xB0`; o frame é considerado válido quando o resultado é zero.

NOTA

As revisões 1.0–1.2 documentavam apenas o CRC8. O CRC3 do reply frame foi identificado na implementação de referência (seção 9.1) e é obrigatório para validar confirmações de escrita.

6.4 Sequência de Inicialização

A inicialização atribui NODE_IDs sequencialmente, começando pelo IC mais próximo ao transceiver:

PASSO	AÇÃO	FRAME (HEX)
1	Atribuir NODE_ID = 0x01 ao 1º IC	1E 80 36 0001 ED
2	Atribuir NODE_ID = 0x02 ao 2º IC	1E 80 36 0002 CA
3	...	...
N	Atribuir NODE_ID = N ao último IC (FN=1)	1E 80 36 08xx CRC

NOTA

O último IC da cadeia deve ter o bit FN (Final Node) setado no campo DATA (bit 11 = 1, ex: `0x0804` para NODE_ID=4). Sem isso, o broadcast gera timeout na iso UART. O watchdog do TLE9012 deve ser alimentado durante a inicialização, caso contrário o IC volta para sleep e perde o NODE_ID.

6.5 Leitura de Tensões

1. Configurar `PART_CONFIG` (0x01) com as células ativas: `0x0FFF` para 12 células
2. Escrever `MEAS_CTRL` (0x18) com PCVM_START=1, CVM_MODE=16-bit: `0xE021`
3. Aguardar tempo de conversão (~2 ms para 16-bit)
4. Ler registradores PCVM_0 (0x19) a PCVM_11 (0x24)
5. Converter: `V_cell [V] = (5.0 / 65536) × RESULT`

FÓRMULA DE CONVERSÃO

$$V = (FSR / 2^{16}) \times RAW$$

Onde FSR = 5,0 V (Full Scale Range do ADC do TLE9012). Para PCVM_0 = `0xABCD`: $V = (5,0 / 65536) \times 0xABCD = 3,355 \text{ V}$.

6.6 Leitura de Temperatura

O TLE9012 suporta até 5 sensores NTC externos, configurados via `TEMP_CONF` (0x04). Os resultados ficam em TEMP_RESULT_0 (0x29) a TEMP_RESULT_4 (0x2D). A conversão depende do NTC utilizado (tabela de lookup específica do componente).

6.7 Portabilidade — Isolamento do Transporte

O módulo `tle9012.c` não deve conter **nenhuma chamada de HAL**. Toda a dependência de hardware é injetada por uma interface de transporte, o que mantém o driver — montagem de frame, CRC, mapa de registradores, conversões — independente do MCU:

```
typedef struct {
    bool (*send)(const uint8_t *buf, uint16_t len);
    bool (*recv)(uint8_t *buf, uint16_t len, uint32_t timeout_ms);
    void (*delay_us)(uint32_t us);
} tle9012_transport_t;
```

Consequência prática: migrar para um AURIX TC375 no futuro exige reimplementar apenas essas três funções sobre ASCLIN, sem tocar na lógica do driver. O mesmo vale para testes unitários em PC, onde o transporte pode ser substituído por um *mock* que reproduz respostas conhecidas do TLE9012.

6.8 Configuração do Periférico no STM32CubeMX

USART1 — Parameter Settings

CAMPO	VALOR	JUSTIFICATIVA
Mode	Asynchronous	TX <code>PA9</code> , RX <code>PA10</code> — casa com os pads LS_TX/LS_RX da adapter board
Baud Rate	2000000	Mínimo funcional é 1 Mbit/s; 2 Mbit/s é o oficial
Word Length	8 Bits	Sem paridade
Parity / Stop Bits	None / 1	—
Data Direction	Receive and Transmit	—
Over Sampling	16	A 170 MHz resulta em <code>BRR = 85</code> exato
Advanced → MSB First	Enable	Obrigatório — protocolo é MSB-first (seção 6.2)
Advanced → Auto Baudrate	Disable	—
Advanced → TX/RX Swap	Disable	—

Alternativa: modo *Single Wire (Half-Duplex)*, que usa apenas `PA9` em open-drain bidirecional. Nesse caso o resistor série fica por conta do projetista. Ambos os modos produzem eco — a diferença é apenas o número de fios.

USART1 — DMA Settings

REQUEST	DIRECTION	MODE	PRIORITY	DATA WIDTH
USART1_TX	Memory → Peripheral	Normal	High	Byte / Byte
USART1_RX	Peripheral → Memory	Circular	Very High	Byte / Byte

Increment Address: apenas **Memory** marcado (default do CubeMX, correto).

NOTA — POR QUE CIRCULAR NO RX

O protocolo é request/response e o host recebe o próprio eco. Em modo *Normal*, o DMA precisaria ser re-armado entre frames, e a 2 Mbit/s a resposta do escravo pode chegar exatamente dentro dessa janela. Em modo *Circular* a recepção nunca para, eliminando a janela de perda. O DMAMUX do STM32G4 roteia os requests automaticamente — nada a configurar manualmente.

NVIC Settings

- **USART1 global interrupt: Enable** — necessário para detecção de linha ociosa (`HAL_UARTEx_ReceiveToIdle_DMA`), que delimita o fim do frame

- `DMA1 channelX global interrupt` — habilitados automaticamente ao adicionar os requests; apenas conferir

Demais periféricos

ITEM	CONFIGURAÇÃO	FASE
Clock	HSI16 → PLL (M=4, N=85, R=2) = 170 MHz ; Flash 4WS + PWR boost mode	1
USART2	<code>PA2</code> / <code>PA3</code> , 115200, assíncrono, sem DMA — VCP do ST-LINK para <code>printf</code>	1
Watchdog do TLE9012	Sem periférico dedicado — SysTick + timer por software	1
FDCAN1	Adiar; exige transceiver externo	3

ATENÇÃO

O watchdog interno do TLE9012 **reseta o `NODE_ID`** se não for alimentado dentro do intervalo configurado (`WDOG_CNT.WD_CNT`), inclusive durante a própria inicialização da cadeia. O tick de *watchdog kick* precisa existir desde a fase 1, antes mesmo de qualquer leitura de tensão.

NOTA — CAMINHO PARA ASIL

Caso o BMS evolua para o AMS definitivo do veículo e seja exigido nível de integridade de segurança (lockstep, ASIL-D), o AURIX TC375 volta a ser candidato. A decisão pode ser adiada sem custo desde que o isolamento de transporte descrito acima seja respeitado desde a fase 1.

7. ESTRUTURA DE DIRETÓRIOS DO PROJETO

```
bms_tle9012/
+-- docs/
|   +-- relatorio_estrutura_software_bms.html
|   +-- relatorio_estrutura_software_bms.pdf
|   +-- infineon-...-usermanual-en.pdf
|
+-- bootloader/                (projeto STM32CubeIDE separado)
|   +-- Core/Src/
|   |   +-- main.c             bootloader main
|   |   +-- can_boot.c/h       protocolo CAN de atualizacao
|   |   +-- flash_if.c/h       interface flash (erase/write/verify)
|   +-- STM32G491RETX_FLASH.ld linker: ORIGIN = 0x08000000, LENGTH = 16K
|
+-- app/                        (projeto STM32CubeIDE principal)
|   +-- Core/Src/
|   |   +-- main.c
|   |   +-- tle9012.c/h
|   |   +-- tle9015.c/h
|   |   +-- crc_j1850.c/h
|   |   +-- params.c/h
|   |   +-- protection.c/h
|   |   +-- balancing.c/h
```

```

| | +-- can_app.c/h
| | +-- bsp.c/h
| +-- STM32G491RETX_FLASH.ld    linker: fases 1-3 -> ORIGIN = 0x08000000
|                                fase 4    -> ORIGIN = 0x08004000, LENGTH = 480K
|
+-- tools/
    +-- flash_tool.py            script Python para envio de firmware via CAN
    +-- param_tool.py            script Python para configurar parametros via CAN

```

8. FLUXO DE ATUALIZAÇÃO EM CAMPO

8.1 Atualização de Firmware (via bootloader CAN)

1. Conectar adaptador CAN-USB ao barramento (ex: PCAN-USB, CANable)
2. Executar `flash_tool.py --port CAN0 --file app.bin`
3. O script envia comando ENTER_BOOTLOADER → a aplicação seta flag em RAM e reseta
4. Bootloader apaga, grava e verifica a nova imagem
5. MCU reinicia na nova aplicação

8.2 Atualização de Parâmetros (sem bootloader)

1. Executar `param_tool.py --port CAN0 --config params.json`
2. O script serializa a struct e envia via mensagens CAN segmentadas
3. A aplicação grava na flash e reinicia com os novos valores

VANTAGEM

Zero recompilação

Para mudar de 12 células para 8, ou de OV 4,2 V para 4,15 V, basta editar o `params.json` e rodar o script. Não é necessário IDE, compilador ou programador.

9. VALIDAÇÃO EM BANCADA

A fase 1 foi levantada em bancada em 2026-08-26, com bateria de 12,67 V no ladder resistivo da placa de avaliação. As 12 células leram **1056 mV** cada, consistente com 12670/12. Isso validou em hardware, de uma vez: baudrate de 2 Mbit/s, ordem MSB-first, formato dos frames, CRC8 e CRC3, atribuição de NODE_ID, mapa de registradores e a fórmula de conversão.

As sete armadilhas abaixo custaram tempo real de bancada. Nenhuma delas é óbvia na documentação, e várias produzem sintomas que apontam para o lugar errado.

9.1 Alimentação: VS e VIO são trilhos independentes

A ARMADILHA MAIS CARA

VS (pino 34) alimenta o núcleo do TLE9015. **VIO** (pino 30) alimenta **exclusivamente a interface UART** e define seus níveis lógicos. São independentes e **ambos obrigatórios**.

Com o `VIO` flutuando, a Figura 4 do datasheet é explícita: `V_VIO < V_VIO_th_UV` (2,2 a 2,76 V) → “UART interface is deactivated”.

Sintoma observado: o transceiver fica vivo do lado da iso UART e surdo do lado do MCU. O sistema não sobe sozinho após ciclar a energia do transceiver e só volta ao **desplugar e replugar o cabo iso UART** — porque isso gera transiente no único lado que ainda funciona. Nenhuma correção de firmware resolve, porque o firmware fala justamente pela interface desativada. E o comportamento é **intermitente**, não morto: pino de alimentação solto flutua perto do limiar por caminhos parasitas.

PINO	FAIXA FUNCIONAL	ALIMENTAR COM
VS (TLE9015)	4,75 – 45 V	Pino <code>5V</code> da Nucleo
VIO (TLE9015)	3 – 5,5 V	Pino <code>3V3</code> da Nucleo
VS / U12P (TLE9012)	4,75 – 60 V	Fonte isolada no ladder

Alimentar o `VIO` com 3,3 V tem um bônus: os pinos UART passam a chavear exatamente no nível do `PC4` / `PC5`, melhor do que deixar o `VREGOUT` interno alimentá-lo com 3,45 V.

9.2 A ligação transceiver↔sensing é cruzada

O sinal entra numa CSC por uma interface e sai pela outra, então lado igual nos dois extremos não tem caminho. Com o MCU no `UART_LS`, a seção 5.1 do datasheet estabelece que os dados saem por `IFH` — o conector `HS`.

TRANSCIVER	SENSING	VÁLIDO
LS (IFL)	H P / H N	Primary on Top
HS (IFH)	L P / L N	Primary on Bottom — validado
LS	L P / L N	sem caminho
HS	H P / H N	sem caminho

9.3 Watchdog derruba o link sem aviso

O campo `WD_CNT` do `WDOG_CNT` (0x3D) decrementa sozinho. Ao chegar em `0x00`, o TLE9012 **entra em sleep e reseta o NODE_ID para 0**. A partir daí, toda transação endereçada ao nó antigo dá timeout, sem nenhum outro sintoma.

Observado em bancada como: a cadeia sobe, lê corretamente por um tempo, e então `meas_count` congela enquanto `meas_fail_count` sobe indefinidamente — mesmo com toda a alimentação presente. O firmware recarrega o contador a cada ciclo de medição e logo após atribuir o `NODE_ID`.

9.4 Recuperação exige reset explícito

`tle9012_assign_node_id()` escreve no nó 0, que é como se endereça um IC ainda não configurado. Se o TLE9012 retiver o `NODE_ID` de uma sessão anterior, não existe mais ninguém no nó 0 e a reinicialização falha para sempre.

A seção 3.6.2 do manual traz o procedimento: escrever `SLEEP_REG_RESET` (`OP_MODE = 0xC404`), aguardar 4 ms e acordar com um comando de leitura. O dispositivo volta em `NODE_ID = 0`.

9.5 Formato da rajada de multiread

O manual documenta os registradores `MULTI_READ` (0x31) e `MULTI_READ_CFG` (0x32), mas **não descreve o frame de resposta**. O layout abaixo foi determinado observando a rajada real em bancada:

Total: 64 bytes para 12 células

```
[0..3]    eco do pedido de leitura (1E ID 31 CRC)
[4..8]    frame da célula 11  \
[9..13]   frame da célula 10  | 12 frames de 5 bytes:
...       | SYNC | ID | DATA_H | DATA_L | CRC8
[59..63]  frame da célula 0   /  CRC cobre os 4 bytes anteriores
```

NÃO VERIFICADO

A **ordem** das células segue a descrição do campo `PCVM_SEL = 0xC` no manual, “*Result of Cell 11-0*” — decrescente. Com ladder resistivo todas as células leem igual, o que torna ordem certa e invertida **indistinguíveis**. Confirmar com fonte desbalanceada ou células reais antes de confiar: é um erro que só se manifesta quando o pack desbalanceia, que é exatamente quando o BMS precisa acertar.

9.6 Dado obsoleto se passando por atual

DEFEITO PRÓPRIO, CORRIGIDO

A primeira versão do laço de medição retornava no caminho de erro sem tocar no snapshot. Ao desligar a fonte da sensing board, `meas_fail_count` subia mas `cell_mv` continuava exibindo 1056 mV indefinidamente.

Num BMS esse é o pior tipo de defeito: agir sobre tensão obsoleta acreditando que é atual. A correção introduziu `cell_data_valid` e `cell_data_age_ms`. **Nenhuma camada superior pode usar `cell_mv` sem conferir `cell_data_valid` antes** — e isso vale também para o CAN: transmitir leitura de dez minutos atrás como se fosse atual é pior que não transmitir nada.

9.7 Tempo de conversão por polling

O bit `PCVM_START` (`MEAS_CTRL` bit 15, tipo `rwh`) é limpo pelo hardware ao final da conversão — “*Bit cleared if conversion done*”. Fazer polling desse bit substitui o atraso fixo arbitrado, com dois ganhos: a leitura sai assim que fica pronta, e uma conversão que não conclui vira erro explícito em vez de dado lido cedo demais.

10. DIAGNÓSTICO E PROTEÇÕES

10.1 Código de falha

Máscara de bit é ilegível em depurador. A variável `fault_code` é de tipo `enum`, e o Live Expressions do CubelDE exibe o **nome** do enumerado em vez do número. Ao lado, `fault_cell` indica a célula ou canal afetado (`0xFF` quando a falha não é por célula).

A ordem é de **prioridade**, não de ocorrência: a falha mais grave ativa vence. Perda de comunicação vem antes de tudo porque, sem dado, nenhuma outra proteção significa nada. Sobretensão vem antes de subtensão porque sobrecarga tem consequência térmica, enquanto subtensão apenas degrada.

PRIO	CÓDIGO	ORIGEM	SIGNIFICADO
1	BMS_FAULT_CHAIN_DOWN	firmware	Cadeia caiu, sem comunicação
2	BMS_FAULT_DATA_STALE	firmware	Medição obsoleta
3	BMS_FAULT_CELL_OV	hardware	Sobretensão de célula
4	BMS_FAULT_CELL_UV	hardware	Subtensão de célula
5	BMS_FAULT_EXT_OVERTEMP	hardware	Sobret temperatura no NTC
6	BMS_FAULT_INT_OVERTEMP	hardware	Sobret temperatura interna do CI
7	BMS_FAULT_UNDERTEMP	firmware	Subtemperatura — o CI não detecta
8	BMS_FAULT_OPEN_LOAD	hardware	Fio de sensoriamento aberto
9	BMS_FAULT_ADC	hardware	Soma dos PCVM diverge do BVM
10	BMS_FAULT_INTERNAL_IC	hardware	Erro interno do TLE9012
11	BMS_FAULT_REG_CRC	hardware	CRC de registrador
12	BMS_FAULT_PS_SLEEP	hardware	Sleep por erro de alimentação
13	BMS_FAULT_BALANCING	hardware	Corrente de balanceamento fora do previsto

HARDWARE OU FIRMWARE — A DISTINÇÃO IMPORTA

As falhas marcadas **hardware** são detectadas pelo próprio TLE9012 e continuam valendo mesmo se o firmware travar. A **subtemperatura** é a única detectada só em firmware, porque o CI não possui limiar de frio — existe apenas `EXT_OT_THR`, para o lado quente. Uma proteção que depende do firmware rodando é estruturalmente mais fraca, e isso deve ser declarado.

10.2 Bits do GEN_DIAG (0x0B)

BIT	MÁSCARA	CAMPO	SIGNIFICADO
0	0x0001	UART_WAKEUP	status — origem do wake-up
1	0x0002	MOT_MOB_N	status — 0 = PoB, 1 = PoT
2	0x0004	BAL_ACTIVE	status — balanceamento em curso
3	0x0008	LOCK_MEAS	status — medição em curso
4	0x0010	RR_ACTIVE	status — round robin ativo
5	0x0020	PS_ERR_SLEEP	sleep por erro de alimentação
6	0x0040	ADC_ERR	soma dos PCVM diverge do BVM

BIT	MÁSCARA	CAMPO	SIGNIFICADO
7	0x0080	OL_ERR	open load
8	0x0100	INT_IC_ERR	erro interno do CI
9	0x0200	REG_CRC_ERR	CRC de registrador
10	0x0400	EXT_T_ERR	sobret temperatura externa
11	0x0800	INT_OT	sobret temperatura interna
12	0x1000	CELL_UV	resumo de sub tensão
13	0x2000	CELL_OV	resumo de sobretensão
14	0x4000	BAL_ERR_UC	subcorrente de balanceamento
15	0x8000	BAL_ERR_OC	sobrecorrente de balanceamento

NOTA

Os bits 0 a 4 são status de operação, não erro. Ver 0x0010 no GEN_DIAG significa apenas que o round robin está rodando. Por isso o acumulador `fault_latched` mascara com 0xFFE0, guardando só do bit 5 para cima.

10.3 Máscaras por célula

Os registradores `CELL_UV` (0x0C) e `CELL_OV` (0x0D) trazem um bit por célula, nos bits 0 a 11: **bit i = célula i** .

VALOR	SIGNIFICA
0x0001	somente a célula 0
0x0800	somente a célula 11
0x0FFF	todas as 12

Com o ladder resistivo todas as células leem igual, então uma injeção de UV ou OV acende **todas de uma vez**: espera-se 0x0FFF.

Identificação do escravo

Cada CSC da cadeia tem seu próprio `NODE_ID` e seus próprios registradores de diagnóstico — a leitura já é endereçada por nó, então distinguir de qual escravo veio a falha é imediato. A variável `fault_node` registra isso.

LIMITAÇÃO ATUAL

O firmware é **single-slave**: tudo está fixado em `BMS_NODE_ID = 1`. Suportar N escravos exige transformar os snapshots (`cell_mv`, `fault_uv`, `temp_ohms` ...) em matrizes indexadas por nó e iterar a inicialização até o último IC, marcando apenas ele como *final node*. O driver já recebe `node_id` em todas as funções; a mudança é de camada de aplicação.

10.4 Histerese

Os flags `CELL_UV` e `CELL_OV` são do tipo `rocv`: uma vez setados, permanecem até alguém escrever `0` na posição. Ou seja, **quando limpar é decisão do firmware** — e limpar assim que a tensão volta a cruzar o limiar produziria oscilação contínua com a célula parada em cima dele.

UV	seta em UV_THR	->	limpa so acima de	UV_THR + 200 mV
OV	seta em OV_THR	->	limpa so abaixo de	OV_THR - 200 mV

Três propriedades deliberadas:

- **A falha sempre seta no limiar exato.** A histerese atrasa apenas a recuperação, nunca a detecção.
- **Só limpa com** `cell_data_valid = 1`. Dado obsoleto nunca pode justificar sair de um estado de falha.
- **A limpeza é por célula.** A célula 3 pode sair de falha sem apagar a da célula 7 — o que importa num pack real, onde as células divergem.

ARMADILHA: O BIT-RESUMO É SEPARADO

Limpar `CELL_UV` / `CELL_OV` **não** limpa os bits 12 e 13 do `GEN_DIAG`, que são resumos em registrador distinto. Sem limpá-los também, o `GEN_DIAG` fica preso exibindo a falha antiga indefinidamente. Foi exatamente esse o sintoma observado em bancada antes da correção.

10.5 Injeção de falha para demonstração

Seis modos, cicláveis pelo botão azul da Nucleo — o que dispensa depurador durante uma inspeção técnica. Também ajustáveis por `sim_request`.

MODO	COMO A CONDIÇÃO É CRIADA	GEN_DIAG ESPERADO
UNDERVOLTAGE	Limiar de UV movido acima da tensão medida	0x1000
OVERVOLTAGE	Limiar de OV movido abaixo da tensão medida	0x2000
OVERTEMP	Limiar de OT no máximo	0x0400
OPEN_LOAD	Diagnóstico reabilitado com ladder resistivo	0x0080
COMM_LOSS	CI enviado para sleep	—
UNDERTEMP	Limiar de firmware abaixo da leitura	inalterado

DECISÃO DE PROJETO

Injeção real, não flag forjado

Nenhum modo escreve em registrador de diagnóstico. Cada um cria uma **condição** que o TLE9012 detecta pelo caminho normal — o caminho de detecção, os registradores e o intertravamento que desliga o balanceamento são os mesmos de uma falha real. O que muda é a origem da condição, não a detecção.

Forjar os flags acenderia o LED sem provar nada, e não sobreviveria a um avaliador que pedisse para ver o `GEN_DIAG`. Declarar “o limiar foi deslocado, a detecção é real” é mais forte do que alegar falha genuína.

Repare que `UNDERTEMP` deixa o `GEN_DIAG` intacto: é a confirmação visível de que aquela proteção é puramente de firmware.

10.6 Sinalização no LD2

ESTADO DO LED	SIGNIFICADO
Piscando a 5 Hz	Vivo e sem falha
Aceso fixo	Falha ativa
Apagado	Firmware travado

Saúde sinalizada por **pisca**, e não por LED apagado, é deliberado: se apagado significasse “tudo bem”, firmware travado seria indistinguível de operação normal. Um AMS não pode ter estado de falha confundível com estado morto.

A condição de acender cobre mais que os flags do CI: **cadeia caída** e **dado obsoleto** também acendem. Para um AMS, não saber a tensão é tão grave quanto saber que ela está errada.

11. REFERÊNCIAS

- Infineon, *TLE9012DQU, TLE9015DQU — User Manual*, Rev. 1.0, Z8F80064984, 2022-01-24.
- STMicroelectronics, *STM32G4 Series — Reference Manual*, RM0440 (cobre a linha G491).
- STMicroelectronics, *STM32G491xC/xE — Datasheet*.
- STMicroelectronics, *NUCLEO-G491RE — User Manual* (pinagem, solder bridges do VCP).
- SAE J1850, *Class B Data Communications Network Interface* — CRC polynomial.
- STMicroelectronics, *AN2606 — STM32 system memory boot mode*.
- MaxMax-embedded, *TLE9012_Arduino_Lib*, licença MIT.

https://github.com/MaxMax-embedded/TLE9012_Arduino_Lib

9.1 Uso da Biblioteca de Referência

A biblioteca *TLE9012_Arduino_Lib* é uma implementação aberta e funcional do driver, validada em ESP32 e Arduino Uno R4. Ela não é portátil diretamente (depende de `HardwareSerial`), mas serve como referência de alto valor em três pontos:

- Mapa de registradores** — `TLE9012_Makros.h` já contém endereços e máscaras de bitfield transcritos do datasheet, eliminando a etapa mais tediosa e mais sujeita a erro do desenvolvimento.
- Modelo de API** — a separação entre acesso de baixo nível (`readRegisterSingle`, `writeRegisterBroadcast`) e funções de alto nível (`readCellVoltages`, `startBalancing`) é um bom ponto de partida para a interface do nosso `tle9012.c`.
- Tratamento de erros por callback** — `attachErrorHandler()` com ponteiros de função por tipo de falha, aderente à abordagem de transporte injetável da seção 6.7.

NOTA — LICENÇA

A licença MIT permite reutilização e modificação, inclusive em projeto fechado, desde que o aviso de copyright e o texto da licença sejam preservados. Se trechos forem portados, manter a atribuição no cabeçalho do arquivo correspondente.

A própria biblioteca documenta que o comando *multiread* não está implementado nas versões atuais. Como o multiread é o mecanismo mais eficiente para ler as 12 tensões num único ciclo, essa funcionalidade foi escrita do zero — ver seção 10.3.

Correção: as revisões 1.0–1.3 citavam `MULTI_READ` em 0x24 e `MULTI_READ_CFG` em 0x25. **Ambos estavam errados** — 0x24 é o `PCVM_11`. Os endereços corretos, conforme a Tabela 27 do manual, são **0x31** e **0x32**.